

PAPO-VLA: Planning-Aware Policy Optimization for Vision-Language-Action Models

Peizheng Guo^{1,2*}, Jingyao Wang^{1,2*}, Changwen Zheng^{1,2}, Wenwen Qiang^{1,2†}

¹Institute of Software Chinese Academy of Sciences,

²University of Chinese Academy of Sciences

{guopeizheng2025, wangjingyao2023, changwen, qiangwenwen}@iscas.ac.cn

Abstract

Vision-Language-Action (VLA) models show promising ability in language-guided robotic tasks. However, making VLA policies reliable remains challenging, because a manipulation task is completed through closed-loop interaction, where each action affects subsequent execution. To analyze this problem, we revisit VLA policy during execution and argue that a VLA policy acts both as a planner, which makes task-oriented decisions that change the direction of execution, and as an executor, which realizes these decisions through dense continuous actions. This view suggests that improving VLA reliability requires particular attention to planning actions. Existing optimization methods can imitate actions or improve complete trajectories, but they usually do not explicitly identify planning actions or measure their importance for task success. To address this issue, we propose Planning-Aware Policy Optimization for VLA models (PAPO-VLA). PAPO-VLA first identifies planning actions by jointly considering action variation and trajectory outcome, then estimates their importance through causal sufficiency and causal necessity, and finally incorporates this importance into GRPO advantage estimation. In this way, more important planning actions receive stronger optimization emphasis, while the whole trajectory is still optimized by trajectory-level feedback. Experiments on multiple benchmarks demonstrate the effectiveness of PAPO-VLA.

1 Introduction

Vision-Language-Action (VLA) models have shown strong potential in various language-guided robotic tasks by mapping visual observations and language instructions into executable robot actions [32, 26, 9, 2, 13]. However, making VLA policies reliable remains challenging. A manipulation task is completed through closed-loop interaction, where each action changes the environment and affects subsequent observations and decisions [13, 20]. Therefore, a reliable VLA policy should not only generate locally plausible actions, but also maintain the whole execution toward task success. This raises a central question: **how to guarantee the reliability of VLA policies in robotic tasks?**

To answer this question, we revisit VLA policy and argue that a VLA policy plays two functional roles (**Figure 1**). First, it acts as a planner when the action changes execution intention, such as switching from approaching an object to grasping it, from grasping to moving it, or from moving to releasing it. These actions determine the direction of the following execution. Second, it acts as an executor when it produces dense actions to connect, adjust, and realize these decisions under changing observations. Both roles are necessary for successful manipulation. Nevertheless, they influence reliability in different ways. Execution actions ensure motion continuity, while planning actions determine whether the following execution is built on a correct task-oriented decision. If a planning

*These authors contribute equally

†Corresponding author

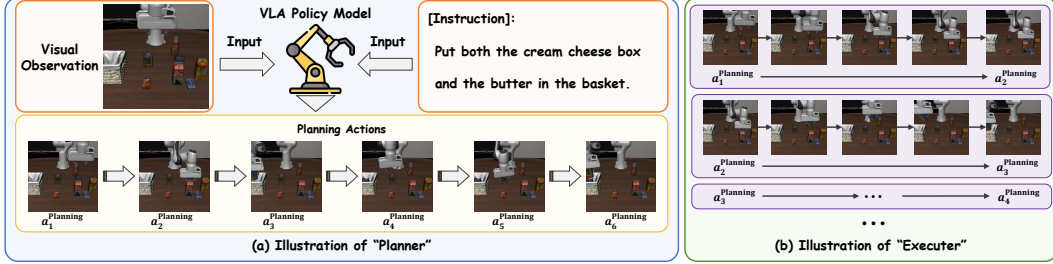


Figure 1: Illustration of Planner and Executor. (a) Planner, which consists of planning actions that are more closely related to task success; (b) Executor, which connects and realizes these planning actions through continuous motions.

action is unreliable, such as grasping at an unstable pose or releasing before proper alignment, later execution may become difficult to recover. Thus, improving the reliability of a VLA policy requires particular attention to the reliability of its planning actions.

This perspective leads to two key problems. The first problem is how to obtain planning actions in a dense execution trajectory. In this work, we seek a way to identify planning actions from the trajectory. Since these actions often appear when the policy changes its execution intention, action variation provides a useful cue for locating planning actions [22, 10]. However, action variation alone is not sufficient. An abrupt action change may correspond to a meaningful transition, but it may also result from unstable control or a failed attempt [21, 11]. Therefore, the planning actions should also be associated with trajectory outcomes, so that the optimization focuses more on transitions that are likely to support successful execution. The second problem is how to measure the importance of these planning actions for task success. A planning action is more important if preserving it helps the subsequent execution move toward success, and if perturbing it makes the task outcome worse. Thus, we need to identify planning actions and estimate their importance for task success.

Based on this analysis, we propose Planning-Aware Policy Optimization for VLA models (PAPO-VLA). Our method first identifies planning actions by action variation and trajectory outcome. This allows policy optimization to focus on actions that indicate execution transitions and appear in trajectories with better outcomes. Then, we estimate the importance of each planning action through causal sufficiency and necessity [18]. Causal sufficiency measures whether preserving the action helps the subsequent execution reach a successful outcome. Causal necessity measures whether perturbing the action makes the outcome worse. An action receives high importance when it is both sufficient for supporting successful progress and necessary for preserving task success. Finally, we incorporate this importance into GRPO advantage estimation, so that more important planning actions receive stronger emphasis while other actions are still optimized by the trajectory outcome.

The main contributions of this paper are summarized as follows: (i) We revisit VLA policy and argue that it plays two functional roles during manipulation: a planner role that determines task-oriented execution decisions, and an executor role that realizes these decisions through dense actions. This view highlights that the reliability of planner is crucial for the reliability of VLA policy. (ii) We propose Planning-Aware Policy Optimization for VLA models (PAPO-VLA), which identifies planning actions, estimates their importance through causal sufficiency and necessity, and incorporates the resulting importance into GRPO advantage estimation; (iii) Extensive experiments conducted on various benchmarks indicate the effectiveness of our method.

2 Related Work

Vision-Language-Action Model. Vision-Language-Action (VLA) models aim to enable robots to generate executable actions in complex environments. Recent studies mainly focus on scaling robotic policies with large pretraining, and exploring post-training to adapt pretrained policies to downstream tasks. OpenVLA [9] unifies open-source vision-language modeling with robot control. $\pi_{0.7}$ [8] uses richer multimodal prompts to support manipulation across tasks, robots, and scenes. RIPT-VLA [23] shows that post-training with task-success rewards can effectively improve the adaptability of pretrained VLA policies in new settings. TGRPO [5] introduces trajectory-wise group relative policy optimization into VLA fine-tuning, providing a direct exploration of GRPO-style post-training for

VLA models. Unlike previous studies, we argue that VLA policy plays two roles during execution: planner and executor. We focus on the reliability of planner to guarantee VLA policy reliability by identify planning actions from dense trajectories, estimate their importance for task success, and incorporate this importance into GRPO-based optimization.

Causality in Robot Learning. Causal theory provides a theoretical framework for modeling interventions, counterfactual changes, and task invariance in robot learning, and has recently been used to improve policy performance. CAIAC [27] identifies state components that are not affected by actions to improve the robustness of offline robot learning. RoCoDA [1] combines causal invariance, geometric equivariance, and data augmentation to improve imitation learning under changing settings. LaVA-Man [31] uses goal-image prediction as a pretraining task to implicitly learn causal relations between visual states and actions in language-guided manipulation. These efforts show that causality can provide more structured modeling principles for robot policy learning, but mainly focus on state factors, data augmentation, or representation learning. We use causality to measure the importance of each planning action during policy optimization. By measuring causal sufficiency and necessity of such planning actions, and incorporating them into optimization, our method enables the optimization to more explicitly distinguish actions that are important for ensuring task success.

3 Problem Setting and Analysis

In this section, we first introduce the problem setting of VLA in **Section 3.1**. Then, we provide the motivation and causal analysis of VLA in **Section 3.2**. More discussion is provided in **Appendix**.

3.1 Problem Setting

In this paper, we consider language-conditioned robotic manipulation, where a VLA receives visual observations and a task instruction, and generates an action sequence to complete the task. Specifically, let l denote the language instruction and o_t denote the visual observation at step t . At each step, the policy takes the most recent H observations and the instruction as input, and outputs an action a_t :

$$a_t \sim \pi_\theta(a_t \mid o_{t-H+1:t}, l), \quad (1)$$

where $o_{t-H+1:t} = \{o_{t-H+1}, \dots, o_t\}$ denotes the observation window up to time step t , and θ denotes the policy parameters. This formulation covers both the case where the policy depends only on the current observation and the case where it conditions on a short history of observations.

The interaction between the policy and the environment forms a trajectory $\tau = (o_0, a_0, o_1, a_1, \dots, o_{T_i-1}, a_{T_i-1})$, where T_i denotes the trajectory length. The corresponding action sequence is $\mathcal{A} = \{a_0, a_1, \dots, a_{T_i-1}\}$.

Following GRPO [6], we optimize the VLA policy by comparing multiple candidate trajectories generated under the same task condition. Specifically, given an initial input $x = (o_0, l)$, the old policy $\pi_{\theta_{\text{old}}}$ samples a group of G candidate trajectories $\{\tau^i\}_{i=1}^G$. Each trajectory τ^i is associated with a scalar trajectory-level reward r^i , which can be computed from task success, trajectory return, or their combination [6]. We optimize the policy by maximizing the following GRPO objective:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim P, \tau^i \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=0}^{T_i-1} \left(\min(\rho_{i,t}(\theta) A_i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) A_i) - \beta D_{\text{KL}}(\pi_\theta \mid \pi_{\text{ref}}) \right) \right], \quad (2)$$

where P denotes the input distribution, $\text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon)$ constrains the update magnitude with clipping threshold ϵ , and $D_{\text{KL}}(\pi_\theta \mid \pi_{\text{ref}})$ denotes the KL divergence between the current policy π_θ and a fixed reference policy π_{ref} . The action-level importance ratio is defined as

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(a_t^i \mid o_{t-H+1:t}^i, l)}{\pi_{\theta_{\text{old}}}(a_t^i \mid o_{t-H+1:t}^i, l)}. \quad (3)$$

The relative advantage A_i measures the relative quality of each trajectory within the group:

$$A_i = \frac{r^i - \text{mean}(r^1, \dots, r^G)}{\text{std}(r^1, \dots, r^G)}, \quad (4)$$

where $\text{mean}(r^1, \dots, r^G)$ and $\text{std}(r^1, \dots, r^G)$ denote the mean and standard deviation of trajectory rewards within the same group, respectively.

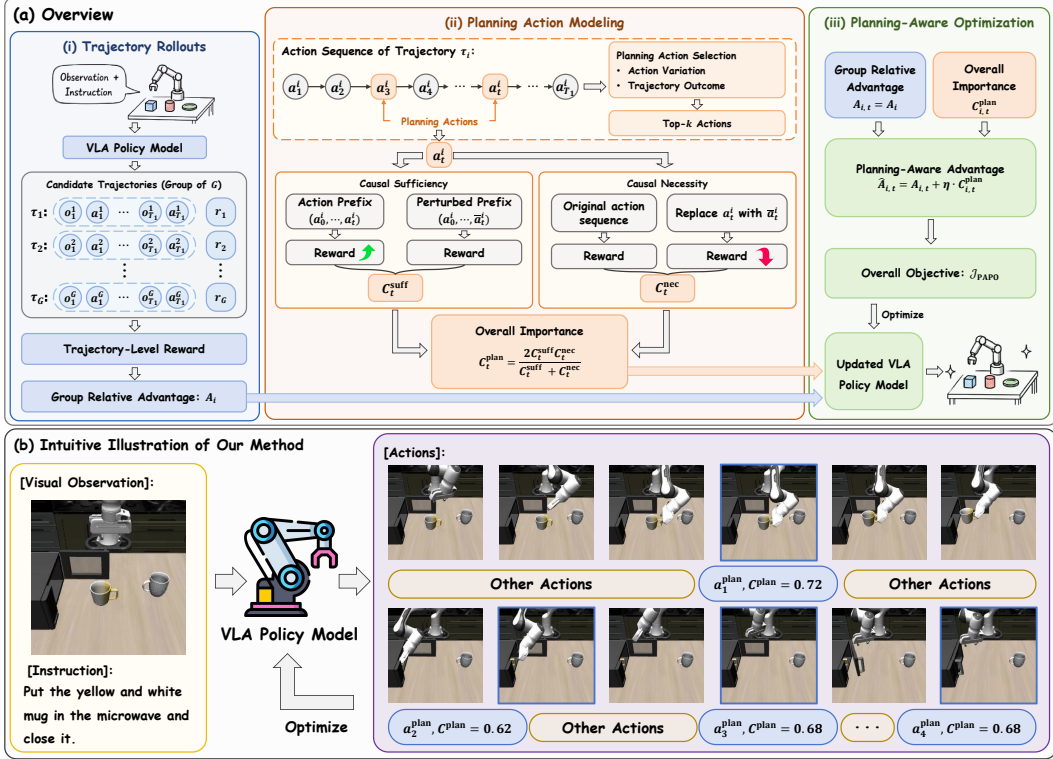


Figure 2: Overview of our proposed method. (a) Overview: Trajectory rollouts are first evaluated by trajectory-level rewards, and planning actions are selected and analyzed through causal sufficiency and necessity to obtain their importance, incorporated into the GRPO advantage estimation to guide optimization. (b) Intuitive Illustration: how different actions in a manipulation trajectory receive different importance to optimize the VLA policy.

3.2 Motivation Analysis

A VLA policy completes a manipulation task through a closed-loop action sequence and each action changes the environment and further affects later observations and decisions [3, 32, 9, 2, 25]. Therefore, the reliability of a VLA policy should not be understood only from whether each action is locally plausible, but from whether the whole execution can be maintained toward task success.

To analyze this problem, we revisit VLA policy during execution. Although the policy is usually formulated as a continuous action generator, it plays two functional roles in a trajectory (**Figure 1**). When the actions change execution intention, such as switching from approaching to grasping, from grasping to moving, or from moving to releasing, the policy acts as a planner. We refer to these task-oriented decision steps as planning actions. When the policy generates dense motions to connect, adjust, and realize these decisions, it acts as an executor. Both roles are necessary, but planning actions have a stronger influence on whether the following execution stays on a successful path.

This makes planning actions important for VLA reliability. If a planning action is unreliable, later execution may become ineffective even if the following actions are smooth. For example in **Figure 1**, an unstable grasp can make subsequent moving actions useless, and an early release can make the remaining trajectory difficult to recover. However, existing optimization methods usually do not explicitly distinguish such actions. Imitation learning reproduces demonstration actions, but does not indicate which actions are more important for task success. GRPO-style optimization uses trajectory-level rewards, but the same trajectory-level advantage may be assigned to all actions in a rollout, making planning actions and ordinary execution actions receive similar emphasis.

This motivates two problems. First, how can we identify planning actions from a dense execution trajectory? Since planning actions often appear when the policy changes its execution intention, action variation provides a useful cue [22, 10, 17, 24]. However, action variation alone is not sufficient, because abrupt changes may also come from unstable control or failed attempts [21, 11]. Thus,

we further consider trajectory outcomes when identifying planning actions. Second, how can we measure the importance of these planning actions for task success? A planning action should be more important if preserving it helps the subsequent execution move toward success, and if perturbing it makes the task outcome worse. Therefore, we use causal sufficiency and necessity to estimate the importance of planning actions and guide policy optimization. Here, we provide the concept of probabilistic causal importance.

Definition 1 (Probabilistic Causal Importance). *Let $X = (O, L)$ denote the policy input, where O is the observation and L is the language instruction, and let $\mathcal{A} = \{a_1, a_2, \dots, a_T\}$ denote the action sequence generated by the VLA policy. Let $E(\mathcal{A}, X) \in \{0, 1\}$ denote the task-completion event under input X , where $E(\mathcal{A}, X) = 1$ means successful task success and $E(\mathcal{A}, X) = 0$ otherwise. For an action a_t , $\bar{\mathcal{A}}_t = (a_1, \dots, \bar{a}_t, \dots, a_T)$ denotes the perturbed action sequence by replacing the t^{th} action with a perturbed action $\bar{a}_t$. Then, the probability that a_t satisfies causal sufficiency and necessity for task success is defined as:*

$$C(a_t) = P_{\text{suff}} P(E(\bar{\mathcal{A}}_t, X) = 0, \bar{\mathcal{A}}_t, X) + P_{\text{nec}} P(E(\mathcal{A}, X) = 1, \mathcal{A}, X), \quad (5)$$

where $P_{\text{suff}} = P(E_{\text{do}(a_t)}(\bar{\mathcal{A}}_t, X) = 1 \mid E(\bar{\mathcal{A}}_t, X) = 0, \bar{\mathcal{A}}_t, X)$ denotes causal sufficiency, i.e., the probability that the task becomes successful when we intervene with $\text{do}(a_t)$ at step t , given that the perturbed sequence $\bar{\mathcal{A}}_t$ would fail. Similarly, $P_{\text{nec}} = P(E_{\text{do}(\bar{a}_t)}(\mathcal{A}, X) = 0 \mid E(\mathcal{A}, X) = 1, \mathcal{A}, X)$ denotes causal necessity, i.e., the probability that the task becomes unsuccessful when we intervene with $\text{do}(\bar{a}_t)$ at step t , given that the sequence $\mathcal{A}$ succeeds.

Definition 1 characterizes the importance of action a_t in task success. P_{suff} measures whether keeping or enforcing this action makes a previously unsuccessful execution more likely to succeed. P_{nec} measures whether perturbing or replacing this action makes a previously successful execution more likely to deteriorate. These two terms correspond to two intuitive questions: whether the action can support success, and whether the action is important for preserving success. Therefore, when an action has both high sufficiency and high necessity, it indicates that the action not only helps drive the task toward successful completion, but also has a substantial influence on the final outcome once it is changed. Such an action should be regarded as more critical to task success, and therefore should receive stronger emphasis in optimization.

4 Methods

Based on the above analysis, we propose Planning-Aware Policy Optimization for VLA models (PAPO-VLA). The core idea is to identify planning actions in trajectories, estimate their importance for task success to guide policy optimization. In **Section 4.1**, we first identify planning actions according to action variation and trajectory outcome. We then measure the importance of these planning actions from causal sufficiency and causal necessity, and aggregate them into an overall planning-action importance. In **Section 4.2**, incorporate the planning-action importance into GRPO advantage estimation, so that actions that are more important for task success receive stronger optimization emphasis. The overview is shown in **Figure 2**, with the pseudo-code in **Appendix**.

4.1 Planning Action Modeling

In this section, we model planning actions in a VLA trajectory. First, we identify planning actions according to action variation and task outcome (**Section 4.1.1**). Then, we evaluate their importance from two causal perspectives. Causal sufficiency measures whether preserving a planning action better supports subsequent successful execution, while causal necessity measures whether perturbing the planning action causes the task outcome to degrade (**Section 4.1.2&4.1.3**). Finally, we combine these two quantities into an overall planning-action importance (**Section 4.1.4**).

4.1.1 Planning Action Identification

Let a trajectory be written as $\tau = (o_0, a_0, o_1, a_1, \dots, o_{T-1}, a_{T-1})$ with the corresponding action sequence $\mathcal{A} = \{a_0, a_1, \dots, a_{T-1}\}$. Planning actions are task-oriented decision steps in the trajectory. They often appear when the policy changes its execution intention, such as switching from approaching to grasping or from moving to releasing. Such actions usually show larger changes compared with neighboring actions. At the same time, a large action change does not necessarily indicate a

planning action, because it may also come from unstable control or a failed attempt. Therefore, we identify planning actions by jointly considering action variation and trajectory outcome.

We first compute the variation magnitude of the t^{th} action:

$$u_t = \begin{cases} \frac{1}{d} \|a_0\|_1, & t = 0, \\ \frac{1}{d} \|a_t - a_{t-1}\|_1, & t > 0, \end{cases} \quad (6)$$

where d is the action dimension, and $\|\cdot\|_1$ denotes the sum of absolute values over all action dimensions, so **Eq. 6** can be understood as the average magnitude of difference between the current action and the previous one across dimensions. A larger u_t indicates a more noticeable action change. To reduce the influence of action scale across trajectories, we normalize it as $\tilde{u}_t = \frac{u_t}{\frac{1}{T} \sum_{j=0}^{T-1} u_j}$.

Then, we introduce a outcome-aware gate derived from the trajectory outcome. Let $g(\tau) = \frac{r(\tau) - r_{\min}}{r_{\max} - r_{\min}} \in [0, 1]$, where $r_{\min}$ and $r_{\max}$ denote the lower and upper bounds of the trajectory reward $r(\tau)$. A larger $g(\tau)$ indicates a better task outcome. Then, the planning-action score is defined as

$$s_t = \tilde{u}_t g(\tau). \quad (7)$$

We then select the top- k actions according to this score and obtain the planning-action mask $m_t^{\text{plan}} = \text{TopKMask}(s_t, k)$. The planning actions of trajectory τ is then represented by the selected action index set $\mathcal{K}_\tau = \{t \mid m_t^{\text{plan}} = 1\}$. This identification prioritizes actions that both show noticeable execution changes and appear in trajectories with better outcomes.

4.1.2 Causal Sufficiency Importance

According to Definition 1, causal sufficiency concerns whether keeping this action can better support the generation of a correct subsequent action sequence and eventually task success. For a planning action a_t , we define the causal sufficiency importance of action a_t as

$$C_t^{\text{suff}} = m_t^{\text{plan}} \cdot \left[\mathbb{E}_{\tau \sim \pi(\cdot | \mathcal{A}_{\leq t})} r(\tau) - \mathbb{E}_{\tau \sim \pi(\cdot | \bar{\mathcal{A}}_{\leq t})} r(\tau) \right]_+, \quad (8)$$

where $[\cdot]_+ = \max(\cdot, 0)$, $r(\tau)$ denote the trajectory-level outcome reward, $\mathcal{A}_{\leq t}$ denote the action prefix keeping the candidate action a_t , and $\bar{\mathcal{A}}_{\leq t} = (a_0, \dots, \bar{a}_t)$ denote the perturbed prefix replacing a_t with a feasible small perturbation $\bar{a}_t$. **Eq. 8** reflects sufficiency for directly comparing the trajectory outcomes of two cases: keeping the current action versus replacing it with a perturbation. If keeping a_t leads to a higher expected reward for the subsequent trajectory, a_t provides more sufficient support for subsequent action generation and positively contributes to task success. Therefore, a larger C_t^{suff} indicates stronger causal sufficiency importance of a_t .

4.1.3 Causal Necessity Importance

Unlike causal sufficiency, causal necessity concerns whether the task outcome deteriorates significantly once the current action is replaced by another feasible action, given that the original action sequence can already support task success. To this end, for a planning action a_t , the causal necessity importance of action a_t can be defined as

$$C_t^{\text{nec}} = m_t^{\text{plan}} \cdot \left[\mathbb{E}_{\tau \sim \pi(\cdot | \mathcal{A})} r(\tau) - \mathbb{E}_{\tau \sim \pi(\cdot | \bar{\mathcal{A}}_t)} r(\tau) \right]_+. \quad (9)$$

where $[\cdot]_+ = \max(\cdot, 0)$, $r(\tau)$ denote the trajectory-level outcome reward, and $\bar{\mathcal{A}}_t = (a_0, \dots, \bar{a}_t, \dots, a_{T-1})$ denote the action sequence obtained by replacing the action a_t with a feasible small perturbation $\bar{a}_t$. **Eq. 9** reflects necessity for directly measuring the expected difference in trajectory outcomes before and after replacing the planning action. If replacing a_t with $\bar{a}_t$ causes a clear drop in the expected reward of the subsequent trajectory, a_t is important for maintaining the task success. Therefore, a larger C_t^{nec} indicates stronger causal necessity importance of a_t .

4.1.4 Overall Planning-Action Importance

After obtaining the causal sufficiency importance C_t^{suff} and the causal necessity importance C_t^{nec} of action a_t , we further aggregate them into a unified quantity to characterize its overall causal

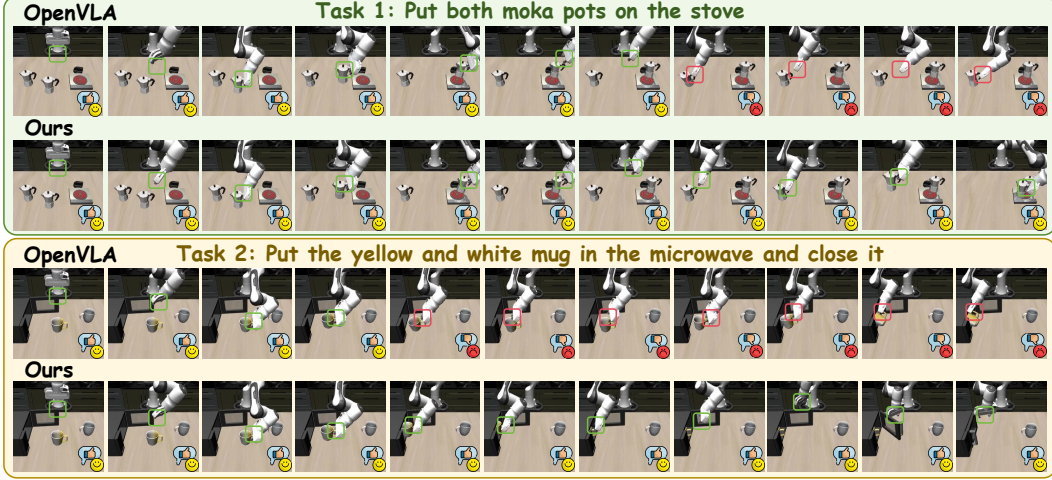


Figure 3: Visualization of comparison between OpenVLA and Ours.

importance to task success. Intuitively, an actual critical action should not only provide sufficient support for successful future behavior, but also cause noticeable degradation once it is replaced. Therefore, a planning action should be regarded as having strong importance when both sufficiency and necessity are simultaneously high. So we define the overall importance of action a_t as

$$C_t^{\text{plan}} = \frac{2C_t^{\text{suff}}C_t^{\text{nec}}}{C_t^{\text{suff}} + C_t^{\text{nec}}}. \quad (10)$$

Eq. 10 provides a characterization of the requirement of being both sufficient and necessary. If an action is high in both sufficiency and necessity, then C_t^{plan} will also be large. In contrast, if the action is high in only one aspect but weak in the other, its C_t^{plan} will be suppressed. Thus, we use C_t^{plan} as the unified planning-action importance for the policy optimization and the causal importance of non-planning positions is explicitly set to zero.

4.2 Planning-Aware Policy Optimization for VLA

In this section, we provide a feasible approach that incorporates planning-action importance into the VLA policy optimization. GRPO is a representative framework for RL, especially in settings where policy quality can be evaluated by verifiable trajectory outcomes [6]. It uses group-wise relative comparison to assess candidate trajectories, which provides a stable optimization signal without requiring additional value modeling. Therefore, we integrate the planning-action importance on the basis of GRPO for VLA optimization, constructing PAPO-VLA.

4.2.1 Planning-Aware Advantage Estimation

In **Section 3.1**, GRPO assigns each sampled trajectory τ^i a group-relative advantage A_i , which reflects the overall quality of the trajectory. A_i treats all actions within the same trajectory equally, i.e. $A_{i,t} = A_i$. We incorporate the planning-action importance obtained in **Section 4.1** into the advantage. Specifically, for action a_t^i in trajectory τ^i , let $C_{i,t}^{\text{plan}}$ denote its overall planning-action importance. We then define the planning-aware advantage as

$$\hat{A}_{i,t} = A_{i,t} + \eta \cdot C_{i,t}^{\text{plan}}, \quad (11)$$

where η controls the strength of planning-action importance. **Eq. 11** is no longer determined only by whether a trajectory is globally good or bad, but also by planning actions.

4.2.2 Planning-Aware Policy Optimization

Based on the advantage in **Section 4.2.1**, we construct the optimization objective under the GRPO framework. Following GRPO [6], the overall objective is

$$J_{\text{PAPO}}(\theta) = \mathbb{E}_{x \sim P, \tau^i \sim \pi^* \theta_{\text{old}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=0}^{T_i-1} \left(\min(\rho_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t}) - \beta D_{\text{KL}}(\pi_{\theta} | \pi_{\text{ref}}) \right) \right], \quad (12)$$

where $\rho_{i,t}(\theta)$, ϵ , and β follow the same definitions as in **Section 3.1**. **Eq. 12** retains the strengths of GRPO while explicitly incorporating planning-action importance into policy learning. Actions that are both more sufficient and necessary will exert a stronger effect. The optimization no longer depends only on whether a trajectory is successful, but also focuses on planning actions that are more important for task success.

5 Experiments

In this section, we conduct extensive experiments on various benchmarks to verify the effectiveness of our method. More details, experiments, and analysis are provided in **Appendix**.

5.1 Experimental Setup

Benchmark. We evaluate our method on two benchmarks, LIBERO [14] and RoboTwin2.0 [4], to examine its effectiveness on language-guided robotic manipulation tasks. All experiments are conducted on H100 GPU clusters. We report task success rate as the main evaluation metric.

LIBERO [14] contains four 10-task suites in the LIBERO Franka Panda simulation environment, including LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-Long. Each task provides RGB observations, robot states, task instructions, and delta end-effector actions. For evaluation, each task is tested with 50 held-out rollouts. The policy is trained for 50 episodes before evaluation.

RoboTwin2.0 [4] contains 50 tasks with 731 object instances and introduces comprehensive domain randomization, including variations in clutter, lighting, background, tabletop height, and language instructions. In our experiments, we use the Agilex Piper robotic arm under domain-randomized task settings. Each task is evaluated on 100 held-out test scenarios. We select 12 tasks from RoboTwin2.0 and categorize them into four horizon levels according to their average step counts.

Baseline. We compare with several representative baselines: PackNet [16], MTL [14], ATM [28], Octo [26], OpenVLA [9], OpenVLA-OFT [13], RDT [15], TraceVLA [30], GRAPE [29], π_{fast} [19], Nora [7], OpenVLA-OFT [13], SFT-4LIBERO [12], MetaVLA [12], and TGRPO [5].

5.2 Results Across Benchmarks

Performance on LIBERO. **Table 1** presents the results on the LIBERO, showing that our method performs well across different tasks. Among them, we reach a higher success rate on LIBERO-Long, indicating that modeling planning actions can provide useful guidance for policy optimization.

Performance on RoboTwin2.0. **Table 2** shows the performance on RoboTwin2.0 across short-horizon, medium-horizon, and long/extra-long-horizon task groups. Our method performs well on both short tasks and tasks that require longer and more structured execution. Additionally, PAPO-VLA shows advantages on long and extra-long tasks, indicating that planning actions modeling

Table 1: Main results on the LIBERO benchmark.

Model	Spatial	Object	Goal	Long	Avg.
PackNet [16]	0.63	0.60	0.75	0.25	0.56
MTL [14]	0.83	0.54	0.80	0.48	0.66
ATM [28]	0.69	0.68	0.78	0.39	0.63
Octo [26]	0.78	0.85	0.84	0.51	0.75
OpenVLA [9]	0.85	0.88	0.79	0.53	0.76
OpenVLA-OFT [9]	0.91	0.95	0.90	0.86	0.91
TraceVLA [30]	0.85	0.85	0.75	0.54	0.75
GRAPE [29]	0.88	0.92	0.83	0.57	0.80
SFT-4LIBERO [12]	0.85	0.87	0.77	0.55	0.76
MetaVLA [12]	0.88	0.88	0.79	0.55	0.78
TGRPO [5]	0.90	0.92	0.81	0.59	0.81
Nora [7]	0.92	0.95	0.89	0.74	0.87
Ours	0.93	0.98	0.98	0.94	0.96

Table 2: Main results of different VLA models on RoboTwin2.0, organized by task horizon.

Model	Short Horizon Tasks (100–130 Steps)				
	Lift Pot	Beat Hammer Block	Pick Dual Bottles	Place Phone Stand	Avg.
π_0 [2]	51.0	59.0	50.0	22.0	45.5
RDT [15]	45.0	22.0	18.0	13.0	24.5
OpenVLA-OFT [13]	10.1	28.1	29.7	17.1	21.3
Ours	62.7	78.5	60.2	33.1	58.6

Model	Medium Horizon Tasks (150–230 Steps)				
	Move Can Pot	Place A2B Left	Place Empty Cup	Handover Mic	Avg.
π_0 [2]	41.0	38.0	84.0	96.0	64.8
RDT [15]	33.0	21.0	42.0	95.0	47.8
OpenVLA-OFT [13]	28.1	37.5	77.3	45.3	47.1
Ours	59.3	41.9	90.1	86.7	69.5

Model	Long and Extra Long Horizon Tasks (280–650 Steps)				
	Handover Block	Stack Bowls Two	Blocks Rank Rgb	Put Bottles Dustbin	Avg.
π_0 [2]	39.0	53.0	45.0	54.0	47.8
RDT [15]	26.0	42.0	17.0	26.0	27.8
OpenVLA-OFT [13]	33.1	40.6	70.2	42.2	46.5
Ours	50.4	69.9	77.2	57.8	63.8

remain effective when needing to maintain correct behavior over extended action sequences. These results demonstrate the effectiveness of our method under diverse task settings and horizon lengths.

5.3 Ablation Study

The effect of different components. We examine three ablated variants: (i) Ours w/o Causal Sufficiency & Necessity (ii) Ours w/o Causal Sufficiency; (iii) Ours w/o Causal Necessity. The results in **Table 3** demonstrate the effectiveness of each component.

Parameter sensitivity. We conduct experiments on the hyperparameter η and search η over $[0, 0.3]$. Results shows that the optimal result is at $\eta = 0.15$, which are our final configuration.

Table 3: The effect of planning-action importance on LIBERO.

Model	Spatial	Object	Goal	Long	Avg.
Ours w/o Suff.&Nec.	0.85	0.88	0.79	0.53	0.76
Ours w/o Suff.	0.89	0.90	0.87	0.80	0.87
Ours w/o Nec.	0.88	0.92	0.89	0.85	0.89
Ours	0.93	0.98	0.98	0.94	0.96

5.4 Case Study and Visualization

To further illustrate the effectiveness, we visualize two representative tasks in **Figure 3**: “putting both moka pots on the stove” and “putting the yellow and white mug in the microwave and closing it”. Our method can produce more complete trajectories and maintain task progress over multiple stages.

6 Conclusion

This paper studies how to improve the reliability of VLA policies in robotic tasks. We revisit VLA execution and argue that a VLA policy plays two functional roles: a planner role that makes task-oriented decisions, and an executor role that realizes these decisions through dense continuous actions. Based on this view, we focus on planning actions, since they determine whether the following execution can remain on a successful path. We propose Planning-Aware Policy Optimization for VLA models (PAPO-VLA), which identifies planning actions from action variation and trajectory outcome, estimates their importance through causal sufficiency and causal necessity, and incorporates the resulting importance into GRPO advantage estimation. In this way, PAPO-VLA preserves trajectory-level optimization while assigning stronger emphasis to planning actions that are more important for task success. Extensive experiments demonstrate the effectiveness of PAPO-VLA.

References

- [1] Ezra Ameperosa, Jeremy A Collins, Mrinal Jain, and Animesh Garg. Rocoda: Counterfactual data augmentation for data-efficient robot learning from demonstrations. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13250–13256. IEEE, 2025.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [4] Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [5] Zengjue Chen, Runliang Niu, He Kong, Qi Wang, Qianli Xing, and Zipei Fan. Tgrpo: Fine-tuning vision-language-action model via trajectory-wise group relative policy optimization. *arXiv preprint arXiv:2506.08440*, 2025.
- [6] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [7] Chia-Yu Hung, Qi Sun, Pengfei Hong, Amir Zadeh, Chuan Li, U Tan, Navonil Majumder, Soujanya Poria, et al. Nora: A small open-sourced generalist vision language action model for embodied tasks. *arXiv preprint arXiv:2504.19854*, 2025.
- [8] Physical Intelligence, Bo Ai, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Greg Balke, Kevin Black, George Bokinsky, Shihao Cao, Thomas Charbonnier, et al. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.
- [9] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [10] George Konidaris, Scott Kuindersma, Roderic Grupen, and Andrew Barto. Robot learning from demonstration by constructing skill trees. *The International Journal of Robotics Research*, 31(3):360–375, 2012.
- [11] Oliver Kroemer, Scott Niekum, and George Konidaris. A review of robot learning for manipulation: Challenges, representations, and algorithms. *Journal of machine learning research*, 22(30):1–82, 2021.
- [12] Chen Li, Zhantao Yang, Han Zhang, Fangyi Chen, Chenchen Zhu, Anudeepsekhar Bolimera, and Marios Savvides. Metavla: Unified meta co-training for efficient embodied adaption. *arXiv preprint arXiv:2510.05580*, 2025.
- [13] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. Simplevla-rl: Scaling vla training via reinforcement learning. *arXiv preprint arXiv:2509.09674*, 2025.
- [14] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [15] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.

- [16] Arun Mallya and Svetlana Lazebnik. Packnet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 7765–7773, 2018.
- [17] Scott Niekum, Sarah Osentoski, George Konidaris, Sachin Chitta, Bhaskara Marthi, and Andrew G Barto. Learning grounded finite-state representations from unstructured demonstrations. *The International Journal of Robotics Research*, 34(2):131–157, 2015.
- [18] Judea Pearl. *Causality*. Cambridge university press, 2009.
- [19] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [20] Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 661–668. JMLR Workshop and Conference Proceedings, 2010.
- [21] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- [22] Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2): 181–211, 1999.
- [23] Shuhan Tan, Kairan Dou, Yue Zhao, and Philipp Krähenbühl. Interactive post-training for vision-language-action models. *arXiv preprint arXiv:2505.17016*, 2025.
- [24] Daniel Tanneberg, Kai Ploeger, Elmar Rueckert, and Jan Peters. Skid raw: Skill discovery from raw trajectories. *IEEE robotics and automation letters*, 6(3):4696–4703, 2021.
- [25] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- [26] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [27] Núria Armengol Urpí, Marco Bagatella, Marin Vlastelica, and Georg Martius. Causal action influence aware counterfactual data augmentation. *arXiv preprint arXiv:2405.18917*, 2024.
- [28] Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. *arXiv preprint arXiv:2401.00025*, 2023.
- [29] Zijian Zhang, Kaiyuan Zheng, Zhaorun Chen, Joel Jang, Yi Li, Siwei Han, Chaoqi Wang, Mingyu Ding, Dieter Fox, and Huaxiu Yao. Grape: Generalizing robot policy via preference alignment. *arXiv preprint arXiv:2411.19309*, 2024.
- [30] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. *arXiv preprint arXiv:2412.10345*, 2024.
- [31] Chaoran Zhu, Hengyi Wang, Yik Lung Pang, and Changjae Oh. Lava-man: Learning visual action representations for robot manipulation. *arXiv preprint arXiv:2508.19391*, 2025.
- [32] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.